A host-side check for an STM32N657 CNN: what matches the PC, and what does not

Alexander Bashirov

15 October 2026

1 Question

The CNN used here is a small MNIST network: two 3×3 convolutions, two max-pools, and two linear layers. The same checkpoint can be run in PyTorch on the PC and on a NUCLEO-N657X0-Q. This note records one fresh on-board measurement and the host-side tools added so that a later export can be checked before it is flashed.

The score is the mean, over images, of the squared Euclidean distance between two length-10 logit vectors. Label accuracy is a different number: the fraction of images whose largest logit equals the written digit. Cosine similarity is a third number: the mean, over images, of the cosine between the two logit vectors. A large squared gap can sit next to a cosine near 1 and next to an unchanged winning class.

2 What was run

The board was reachable on the ST-LINK (`/dev/ttyACM0`). The firmware loaded into RAM is the CNN with FP16 weights and FP32 activations (Helium, weight-only). It is not the Cube.AI graph. The run used the project's usual subset: MNIST test images, `numpy` generator seed 0, 200 images, indices sorted, pixels divided by 255.

On the PC, the same checkpoint was evaluated twice. Once with the stored FP32 weights. Once after `stm32nn.round_fp16`, which casts every parameter to IEEE FP16, nearest, and stores it back in FP32. `stm32nn.compare` then scored the board logits and the Cube.AI FP32 logits, already saved from the earlier serial run of the same 200 images, against the PC FP32 logits.

3 Results

Run	Logit MSE vs PC FP32	Cosine	Argmax differs
PC, weights rounded to FP16	2.422×10^{-5}	1.000	0/200
STM32, Helium, same rounding	2.422×10^{-5}	1.000	0/200
STM32, Cube.AI FP32 graph	5.445	0.998	0/200

Label accuracy is 0.995 on all three runs and on the untouched FP32 network: 199 of 200 digits. The Helium program and the PC rounding differ only in the last digits of the logit MSE (2.422037×10^{-5} against 2.421720×10^{-5}). The file written by the eval script reports 2.422138×10^{-5}. Figure 1 shows that this gap is small on every image.

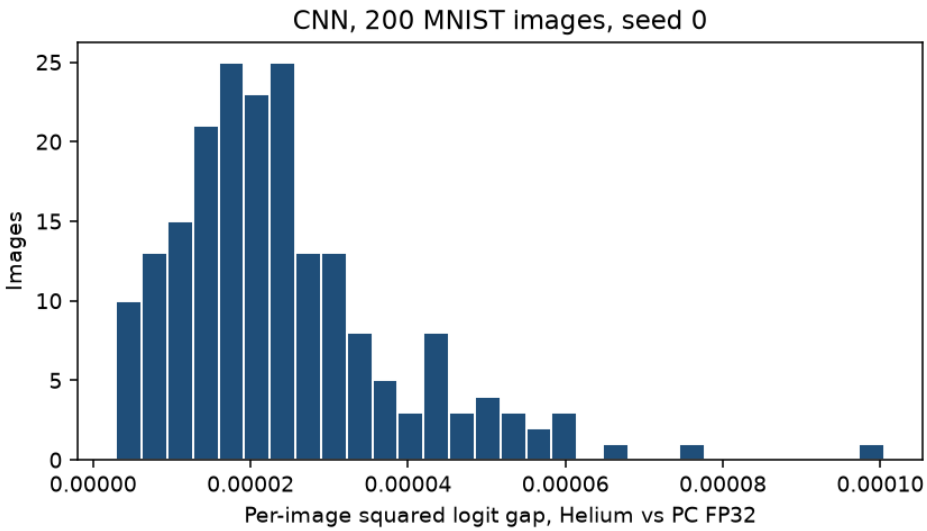


Figure 1: Squared logit gap of the Helium CNN against PC FP32, one bar per image.

The Cube.AI FP32 graph is a different program. Its mean squared gap is about 2×10^5 times larger. Figure 2 is the histogram of the base-10 logarithm of that per-image gap, because the raw values span more than one decade. The cosine stays at 0.998 and the winning class does not change. The squared gap and the class decision are not substitutes.

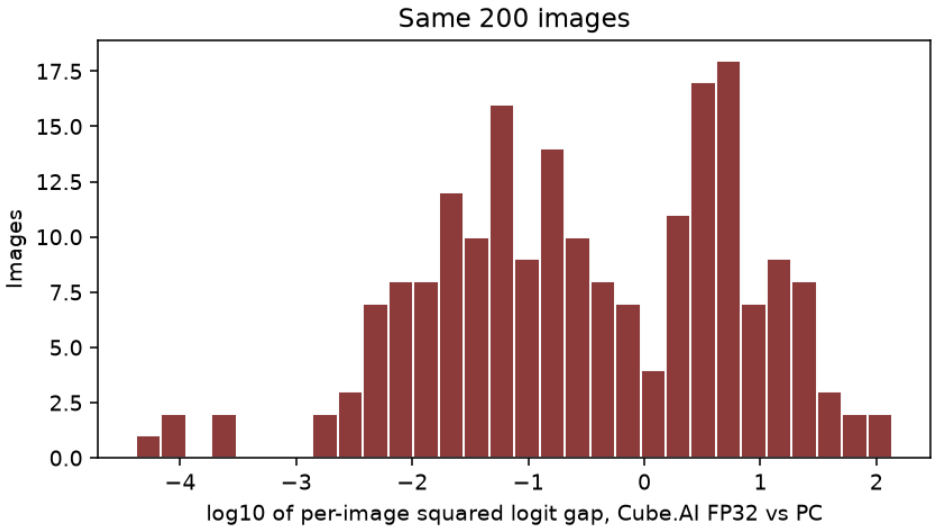


Figure 2: Cube.AI FP32 CNN against the same PC FP32 logits. The axis is logarithmic.

The Helium kernel time, from the firmware cycle counter and excluding the serial transfer, has mean 520.9 ms (Figure 4).

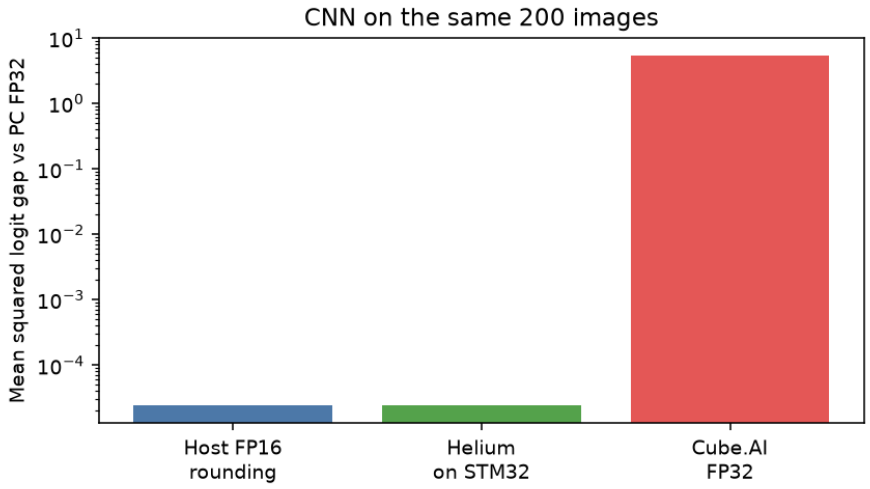


Figure 3: The three gaps on one logarithmic axis. The left two bars are the FP16 rounding. The right bar is the Cube.AI FP32 graph.

4 The host extension

`stm32nn` is a C++ PyTorch module built with CMake. It does not run on the microcontroller and it does not alter the Cube.AI graph. The operations used or added for this check are:

- `round_fp16`: IEEE FP16 nearest, stored back in FP32. This is the rounding in the left bar of Figure 3.
- `to_nhwc`, `to_nchw`: the generated CNN buffers are channel-last. PyTorch is channel-first.
- `quantize_per_channel`: symmetric int8 along one axis. The scale is $\max|w|/127$, and the codes lie in $[-127, 127]$. The returned tensor is FP32. On the first convolution the mean squared change of the weights is 5.05×10^{-7}. That is a weight rounding, not a board measurement.
- `max_pool2d`, plus the existing `conv2d` and `linear`: the generated schedule keeps convolution, ReLU, pool, and linear as separate steps.
- `compare`: logit MSE, argmax disagreement, and mean cosine. Section 3 uses this function.
- `check_export` reports a float16 parameter, a prime channel count above 64, a sample that is not a float image divided by 255, and an ONNX file that stores FP16. On this CNN and a zero 28×28 sample the list is empty. A 257-channel weight and a sample equal to 255 produce both warnings.

The `conv2d` and `linear` loops are a fixed FP32 accumulation written for the extension. They are not the loop inside the Cortex-M55 network runtime library. The generated CNN calls the float convolution inside that library. The body of that function is not shipped as C. Copying a guessed loop into the extension would not make the PC reproduce the Cube.AI number 5.445.

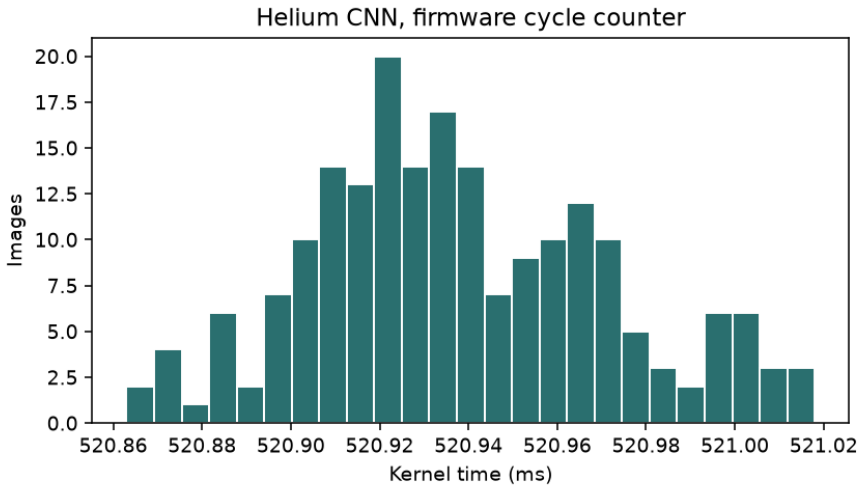


Figure 4: Per-image kernel time of the Helium CNN.

5 What this does not claim

The Helium result is a match between the PC rounding and the board for this one CNN, this 200-image subset, and this firmware. The Cube.AI FP32 number is from the saved serial run of the same subset, not from a new flash tonight. Closing that gap requires the generated C model to be validated on the PC with ST Edge AI, which is not installed here, or a change of the graph that is then flashed and measured again.